# Machine Learning Hw2 Report

學號：N26140888
姓名：黃沭蓁

使用 PyTorch 實作基於 DeepLabV3 搭配 ResNet50 骨幹的語義分割模型，用於處理空拍圖 (UAV) 資料集，並輸出訓練紀錄、視覺化結果與 submission.csv 檔案。

**資料夾結構**

▽ Data and Input Structure

```
.
├── train.py                    # 主程式
└── UAV_dataset/                # Dataset (DATA_DIR)
    ├── train/
    │   ├── imgs/               # training images
    │   └── masks/              # corresponding grayscale segmentation
    └── test/
        └── imgs/               # 1,000 test images (no masks provided)
```

▽ Output Files

```
.
├── best_model.pth              # 最佳模型權重 (以 Val Loss 為準)
├── submission.csv              # 測試集 RLE 結果
├── log/                        # 訓練紀錄
│   └── train_log.csv           # 每個 Epoch 的 Loss 紀錄
└── fig/                        # 視覺化圖表
    ├── training_loss_curve.png # 訓練 Loss 曲線圖
    └── error_analysis_*.png    # 錯誤分析圖 (3張)
```

**train.py 簡介**

| 分區 | 行 | 功能 | 說明 |
|---|---|---|---|
| **Definitions** | 10 | RLE、Denormalize | 實作 RLE 格式轉換<br>輸出圖片時將 Normalized<br>影像還原 |
| **Transform** | 35 | Augumentation | 定義 TrainTransform<br>(Flip + ColorJitter + Norm)<br>TestTransform (Norm) |
| **Dataset** | 97 | 資料讀取與預處理 | 讀取影像和 mask 並 resize 到<br>512 × 512<br>（LINEAR/NEAREST 插 |
| **Model/Train/Evaluation** | 168 | 模型與訓練邏輯 | 建立 DeepLabV3-ResNet50 模<br>（輸出 16 類）、單一 epoch<br>訓練迴圈 |
| **Main** | 230 | 訓練與<br>Early Stopping | 執行訓練主迴圈，包含<br>DataLoader 建立、Adam<br>優化器、StepLR<br>排程器、類別加權 Loss<br>與最佳模型儲存 |
| **損失曲線圖** | 320 | 視覺化訓練 Loss | 輸出訓練 Loss 曲線圖到 ./fig |
| **推論階段** | 334 | 測試集推論與 CSV | 載入最佳模型，對測試集進行推<br>並使用 TTA (<br>Test Time Augmeation：原<br>+ 左右翻轉平均) 後，將結果 re<br>回原圖尺寸並轉成 RLE 輸出<br>submission.csv |
| **錯誤分析** | 387 | 錯誤地圖視覺化 | 隨機抽樣訓練集並輸出 Error M<br>（預測錯誤率 $> 5\%$ 的 3 張影 |

**執行 `train.py`**

```
python train.py
```

### Output Files

| 檔案/路徑 | 說明 |
| --- | --- |
| `best_model.pth` | Val Loss 最小時的模型權重 |
| `submission.csv` | 測試集圖片的 RLE 編碼 |
| `./log/train_log.csv` | 紀錄 Loss 和 LR 變化 |
| `./fig/training_loss_curve.png` | 訓練 Loss 隨 Epoch 變化的圖表 |
| `./fig/error_analysis_*.png` | 3 張模型預測錯誤率較高的訓練集圖 |

# Data Analysis & Preprocessing

## Dataset Exploration

• Describe the dataset structure (train/test split, number of images per class).

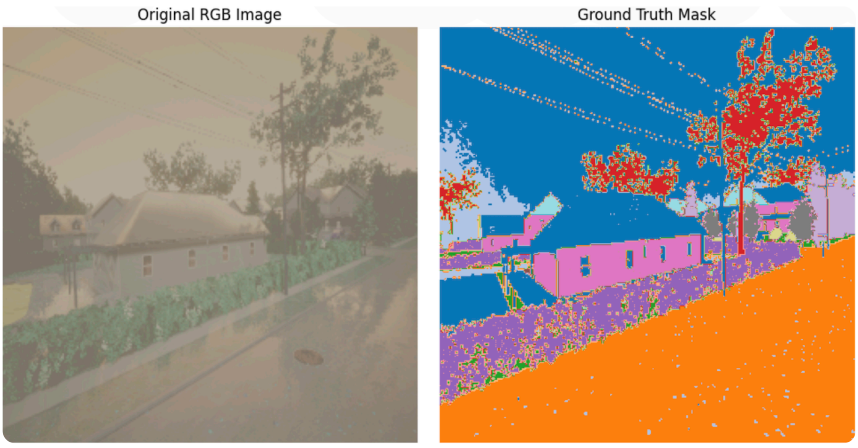
> **Training Set**
> train/imgs：4,000 images in RGB format.
> train/masks: 4,000 grayscale masks corresponding to the training images.
> Each pixel value ranges from 0 to 15, where each integer represents a semantic class.
>
> **Test Set**
> test/imgs: 1,000 images in RGB format.
> No masks are provided. Participants must generate predictions for these images and submit them in RLE format according to the submission rules.

- Provide data visualization (class distribution, example images).

Class Distribution Chart：4000 grayscale masks 類別分佈

分析上圖，像是天空、地面、建築這類經常出現的背景像素，比大部分面積較小物體類別多了 70 倍以上，因此可知有 class imbalance 的狀況。

展示 RGB 影像 與其對應的 真實 Mask (下圖為 785.png)



## Image Preprocessing & Augmentation

• Explain how images were preprocessed (e.g., resizing, normalization, grayscale conversion).

Resizing

- 所有輸入 RGB 影像與 Masks 解析度皆調整為 $512 \times 512$

> `RGB Images` 用 Bilinear Interpolation 雙線性插值保留影像的平滑特徵。`Masks` 用 Nearest-Neighbor Interpolation 最近鄰插值。是為了避免插值運算產生不存在的類別數值（例如 Class 0 和 Class 2 之間不應產生小數點或 Class 1），確保 0-15 的整數標籤保持完整 。

- 選擇 $512 \times 512$ 作為輸入尺寸，是為了在上取得計算效率與顯存利用率的最佳平衡。在這個尺寸下能夠將 Batch Size 設定為 8，在單次迭代中計算更穩定的梯度，有助於加速模型收斂，避免訓練因 CUDA Out of Memory (OOM) 錯誤而中斷。

Normalization

- Method 對所有輸入影像進行 Z-score 標準化，將像素值從 $[0, 255]$ 轉換為標準常態分佈。
- Parameters 使用 ImageNet 數據集的統計數據：
  - Mean `[0.485, 0.456, 0.406]`
  - Std `[0.229, 0.224, 0.225]`
- 選用的模型 DeepLabV3+ 使用了在 ImageNet 上預訓練的 ResNet50 作為 Backbone，使用相同的 Mean 和 Std 對輸入影像正規化。能確保輸入數據的分佈與預訓練權重互相匹配，加速收斂並提升模型 Transfer Learning 效果。

Format Conversion

- RGB Input: 確保資料格式以符合 PyTorch 模型輸入要求。
- Grayscale Masks：分割標籤 (Ground Truth) 被讀取為 Grayscale 模式，其中像素值直接對應 0 到 15 的語意類別。

• Describe any data augmentation techniques used to improve model performance.

為了提高模型泛化能力並減少 Overfitting 的風險，在訓練過程中採用 Online Data Augmentation 策略。所有幾何變換都會確保影像 ($\mathrm{img}$) 和標註 ($\mathrm{mask}$) 同步處理，以保持標籤的準確性。

1. 幾何變換 (Geometric Transformations)

透過改變影像的空間結構來擴充數據多樣性，使模型能夠識別不同視角或方向的目標：

- Random Horizontal Flip
  - 設定： 機率 $p = 0.5$。
  - 以 $50\%$ 的機率對影像和 Masks 進行左右翻轉。這模擬了空拍圖在不同方位下的拍攝效果，確保模型對物體在畫面中的水平位置不敏感。
- Random Vertical Flip
  - 設定：機率 $p = 0.5$。
  - 以 $50\%$ 的機率對影像和 Masks 進行上下翻轉。這模擬了影像的 $180^\circ$ 旋轉，有助於模型適應空拍方向的變化，進一步增強泛化能力。

2. Photometric Distortions

> 僅用於 RGB 影像，模擬真實世界中光照和環境的變化，防止模型過度依賴特定色彩資訊

- Color Jitter
  - 設定：亮度 (brightness=0.3)，對比度 (contrast=0.3)，飽和度 (saturation=0.3)，色相 (hue=0.1)
  - 目的： 隨機調整影像的四個光度參數。模擬了由天氣、陰影或鏡頭變化所引起的色彩和光照不穩定性，例如模擬低對比度的霧氣或色相偏移的沙塵天氣，模型對環境光線變化具有強大的抵抗力。

# Model Selection & Training

## Model Architecture(10%)

• Explain which deep learning model you chose and why (e.g. ResNet, DenseNet).

> 使用 **DeepLabV3** 模型，以在 **ImageNet 上預訓練的 ResNet50** 作為其特徵提取骨幹 。

> 1.**DeepLabV3 的多尺度優勢：ASPP 模組**
>
> UAV 航拍影像的特性是物體尺度差異極大。DeepLabV3 的核心 ASPP (Atrous Spatial Pyramid Pooling) 模組透過結合不同擴張率 (Dilation Rate) 的空洞卷積 (Atrous Convolution)，能同時捕捉多尺度的上下文資訊，大幅提升了模型對小物件和邊緣細節的分割能力，適用於複雜的空拍圖場景。

2.**ResNet50 實現高效遷移學習 Transfer Learning**

> 由於訓練資料集僅有 $4000$ 張影像，從頭訓練一個像 ResNet50 這樣的深層網路容易導致 Overfitting。使用在 $\mathrm{ImageNet}$ 上預訓練的 $\mathrm{ResNet50}$ 權重，利用其已學會的強大通用特徵提取能力（如紋理、邊緣），可以加速模型收斂，提升了模型在有限數據上的泛化效果。

• Describe the architecture details (number of layers, activation functions, optimizers).

**核心架構：DeepLabV3 + ResNet50**

- **Encoder：** 採用 **ResNet50** 作為特徵提取骨幹。
  - **層數：** ResNet50 是一個深層網路，包含數十個卷積層。
  - **激活函數：** 主要使用 ReLU 函數。
- **Decoder：** 包含 **ASPP** 模組，用於融合多尺度上下文資訊，進行特徵圖的上採樣。
- **激活函數：** 模型內部主要使用 **ReLU**，在最終輸出層之前沒有使用 Softmax。

**Output Layer Modification**

- 將原始模型的最終分類頭替換為一層 1×1 卷積層 `nn.Conv2d(256, 16, kernel_size=1)` 。
- 此層將 Decoder 輸出的 $256$ 個特徵通道精確地壓縮為 $16$ 個輸出通道，以精確對應數據集中的 $0$ 至 $15$ 個語義類別。

**Optimizers & Loss**

- Optimizer：用 **Adam** 優化器，初始學習率 $LR = 1e-4$。
- 學習率排程器 Scheduler：用 **StepLR**，設定每 $8$ 個 Epoch，就將學習率乘以 $0.5$ (減半)，幫助模型在訓練後期穩定收斂。
- Loss Function：採用加權交叉熵損失 ($\mathrm{nn.CrossEntropyLoss}$ with weights)。根據各類別像素數的頻率倒數平方根計算權重，處理空拍圖中嚴重的類別不平衡問題。

**Pixel-wise Classification**

1.模型輸出的 $\mathrm{Tensor}$ 形狀為 $(Batch, 16, H, W)$。這代表對於影像中的每一個像素 $(h, w)$，都有一個長度為 $16$ 的向量，表示該像素屬於各類別的信心分數 (Logits)。

2.Inference階段，對 Channel 維度（第 1 維）執行 Argmax 運算。此操作選取分數最高的索引值 ($0$ 至 $15$) 作為該像素的最終預測類別標籤 ($\mathrm{Mask}$)，完成語義分割。

## Loss Function & Optimization

- Justify your choice of loss function (e.g., cross-entropy loss).

**Loss Function**

語義分割是逐像素分類任務，要求模型在 $16$ 個語義類別中為影像中的每一個像素擇一分類，因此選擇 CrossEntropyLoss 作為基礎損失函數。

**Class Imbalance**

訓練資料的 pixel 分佈存在極度嚴重的類別不平衡問題（例如 class 0 佔約 8 千多萬個像素，而 class 15 僅有一百多萬個像素）。如果使用一般的 CrossEntropyLoss，模型會偏向預測佔比最大的類別，導致對少數類別的學習效果極差。

為了解決這個問題，使用 **Inverse Frequency Weighting**，放大了少數類別在損失計算中的影響力，迫使模型更重視學習那些在資料集中稀疏的物體特徵，從而提高整體分割性能。

1.先精確統計出每個類別的 pixel 數量 (counts)。
2.權重：頻率倒數的平方根

$$\text{weights} \propto \frac{1}{\sqrt{\text{counts}}}$$

3.正規化後的權重導入損失函數

```
nn.CrossEntropyLoss(weight=weights.to(device))
```

- Explain the optimizer and learning rate schedule used.

1.Optimizer：Adam

- Code： `torch.optim.Adam(..., lr=1e-4)`
- Adam 是一種高效的適應性學習率算法。它通過追蹤梯度的一階矩二階矩，為模型中的每個參數獨立計算並調整學習率，使得訓練過程更加穩定，適用於複雜的深度卷積網路 (DeepLabV3/ResNet50)，能比傳統 SGD 更快地收斂。
- 初始 Learning rate： $1e-4$。對於使用預訓練權重的遷移學習任務，較小的 LR 是標準做法，可確保在訓練初期避免破壞 ResNet50 已學到的良好特徵。

2.學習率排程器 LR Scheduler：StepLR

- Code： `torch.optim.lr_scheduler.StepLR(optimizer, step_size=8, gamma=0.5)`
- $\text{step_size} = 8$ 表示每訓練完 $8$ 個 $\text{Epoch}$ 執行一次學習率調整，$\text{gamma} = 0.5$ 表示每次調整時，將當前的學習率乘以 $0.5$ (即減半)。這種階梯式衰減策略允許模型在訓練初期快速探索損失曲面，隨著 $\text{LR}$ 逐步降低，模型能夠在訓練後期以更小的步長進行微調，避免震盪，更精準地收斂到損失函數的局部極小值，從而提高最終模型的性能。

## Training Process & Hyperparameters

• Describe the batch size, number of epochs, validation split.

```
BATCH_SIZE = 8  # Batch size
```

```
EPOCHS = 30     # Numbers of epochs
```

```
patience = 5    # Early Stopping：連續 5 個 epoch 沒進步就停止
```

Validation split

train_subset = 4000×0.8：3200 張影像用於模型參數更新。
val_subset = 4000×0.2：800 張影像用於在每個 Epoch 結束後評估模型的泛化能力和決定最佳權重。

```
train_size = int(len(full_dataset) * 0.8)
val_size = len(full_dataset) - train_size
```

• Discuss any techniques used to prevent overfitting (e.g., dropout, batch normalization).

1.**Transfer Learning**

- 使用 ImageNet 預訓練權重初始化 DeepLabV3 的 ResNet50 骨幹。
- 由於訓練數據 4000 張，規模有限，遷移學習能利用預訓練模型已學會的通用特徵，大幅減少從頭訓練所需的數據量，降低在小型數據集上產生過度擬合的風險。

2.**Early Stopping**

- 設定 Patience 為 **5** 的早期停止機制。訓練期間會追蹤驗證集損失 (Val Loss)，並在 Val Loss 連續 5 個 Epoch 未能改善時，自動終止訓練。
- 確保模型在泛化能力最佳的時刻停止訓練，避免模型在訓練後期開始過度擬合訓練集。

3.**Data Augmentation**

- 在訓練集上採用線上隨機增強，包括隨機水平翻轉、隨機垂直翻轉以及 Color Jitter 調整亮度、對比度、飽和度、色相。
- 擴充訓練數據的多樣性，強迫模型學習對幾何方向和光照變化的 Robustness，從而提高其對未見數據的泛化能力。

4.**Batch Normalization**

- 利用 ResNet50 骨幹中內建的 Batch Normalization 層來穩定訓練過程。
- BN 層對每個批次的特徵分佈進行標準化，這除了加速訓練外，還引入了輕微的**正規化**效果（由於每個批次的統計數據不同），有助於提升模型的泛化能力。

5.**Validation Split**

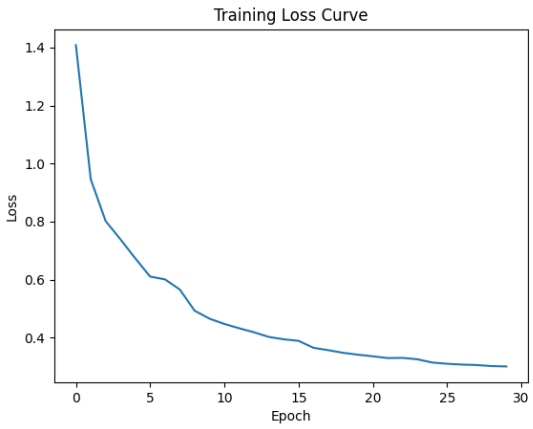
- 將原始數據集以 80% 訓練集 / 20% 驗證集 (3200 / 800) 的比例進行隨機切分。
- 驗證集不參與梯度更新，僅用於獨立評估模型對未見數據的真實泛化能力，是過擬合監控的基礎。

# Experiment & Results

## Comparison of Different Approaches(10%):

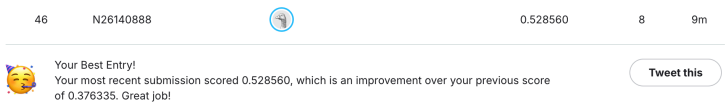
- If multiple models were tested, compare their performances.

Training Loss Curve




上圖顯示出 Loss 隨著 Epochs 上升而下降，證明模型有在學習。

Leaderboard mIoU Score：**0.528560**




- Discuss strengths and weaknesses of your chosen model.

**Strengths**

1.**多尺度上下文感知 (ASPP)**
DeepLabV3 的核心 ASPP 模組透過不同擴張率的空洞卷積，能同時捕獲多種尺度的特徵。這對於空拍圖中物體尺度差異極大的場景至關重要，顯著提升了對小物件與邊緣的分割準確性。

2.**高效的特徵提取與遷移學習 (ResNet50)**
使用在 ImageNet 上預訓練的 ResNet50 權重，模型可以利用已學會的通用特徵。這使得模型在僅有 4000 張影像的中小型數據集上能夠快速收斂，並保持強大的泛化能力，有效降低了過擬合的風險。

3.**解決梯度消失問題**
ResNet50 採用了殘差連接 (Residual Connections)，確保在訓練這個深層網路時梯度能夠順利回傳，使得訓練過程更加穩定和高效。

4.**改進的邊緣輸出**
相比於更早期的全卷積網路 (FCN)，DeepLabV3 通過融合多層次的特徵，在分割物體的邊緣細節處理上通常能提供更優異的表現。

**Weaknesses**

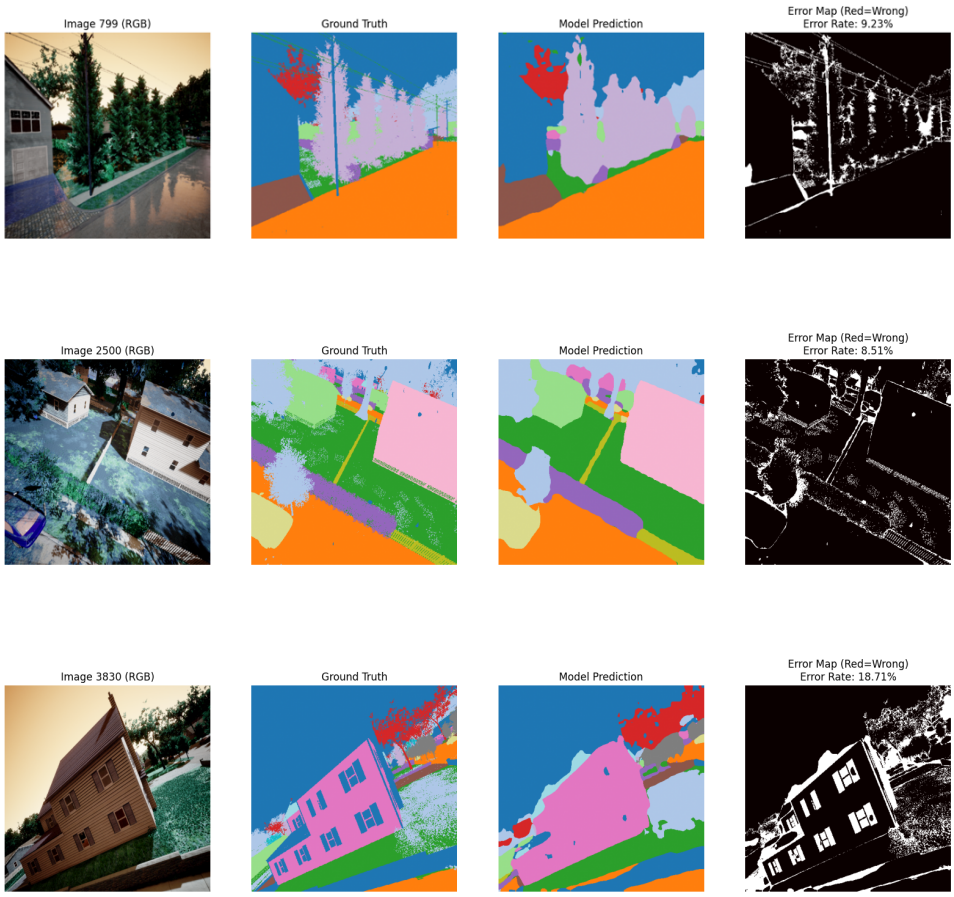
1.**計算複雜度高**
ResNet50 骨幹和 ASPP 模組的參數量和計算量相對較大。這使得模型的訓練和推論速度不如輕量級模型，對 GPU 資源、顯存的需求也更高。

2.**小物件資訊丟失**
儘管 ASPP 有助於多尺度感知，但模型在高倍數下採樣的 Encoder 階段，仍可能導致空拍圖中極小目標（例如遠處的單一目標）的特徵資訊被稀釋或丟失，進而影響其分割效果。

3.**邊緣仍有模糊空間**
在將低解析度的特徵圖上採樣回最終輸出尺寸時，模型仍可能產生不夠清晰的邊緣或輕微的鋸齒狀分割結果，尤其是在複雜的幾何邊界上。

## Error Analysis & Model Improvements(5%):

- Identify common misclassifications and suggest ways to improve them.




三張模型預測錯誤率較高的的 Mask 圖片。

**錯誤分析**

第一張影像中，出現相鄰的樹木邊界的誤判，樹葉的輪廓也不太明顯，segmentation 變得模糊，在樹木前方的電線桿沒有被判斷出來。

第二張影像中，錯誤集中在屋頂、牆面與樹木的交界處。地面顏色不均勻的細節沒有被辨識出來。在光線偏強或顏色飽和度較高的區域，預測結果會變得更大片化，無法保留原始結構。

第三張影像的錯誤情況最嚴重。建築物形狀不明確，道路不筆直，模型忽略了窗戶細節。後方的建築物、樹木與背景交界被混淆成粗糙的分類，是錯誤率最明顯的。

**總結**

綜合以上可看出三類主要問題：邊界模糊導致的輪廓錯誤、光線或顏色變化造成的區域混淆、還有大角度旋轉或透視造成的物體形變。改善方式可以加入更多 Augumentation（旋轉、亮度變化、陰影、隨機裁切）、提高訓練解析度、使用更強的邊界感知損失函數，以及採用對細節較敏感的模型架構。如此便可有效降低錯誤率，改善模型對複雜場景的辨識能力。

• Discuss possible improvements (e.g., using pre-trained models, tuning hyperparameters).

**Problem**

1.Boundary Errors 邊緣與細節模糊

建築物、道路和不同植被類型之間的邊界切割不精確，結果可能呈現鋸齒狀或模糊。

DeepLabV3 在 Encoder 階段進行多次下採樣，雖然有助於上下文感知，但在上採樣回最終尺寸時，邊緣的空間細節資訊被稀釋。

2.Small Object Errors 小物件遺失

遠處的車輛、狹窄的橋樑或交通標線等小目標類別被錯誤分類為周圍的大面積背景。

小目標特徵在深度網路的連續卷積和池化中迅速消失。儘管使用了加權 Loss，但極少數像素的特徵仍難以在低解析度特徵圖中被有效保留。

3.Ambiguous Class Errors 相似類別混淆

模型難以區分視覺上高度相似的類別，例如人工路面與非人工路面，或顏色相似但尺寸不同的低矮建築與高層建築，僅依賴局部紋理或顏色難以區分，模型需要更廣泛的上下文資訊才能做出準確判斷。

**Improvements**

1.提升邊緣精確度

- 升級模型架構：如果資源允許可升級至 **DeepLabV3+**。該模型增加了額外的 Decoder 模組，專門用於融合 Encoder 輸出的低階特徵，有助於恢復空間細節並優化邊界。
- 增加輸入尺寸： 嘗試將輸入尺寸從 $\mathbf{512} \times \mathbf{512}$ 提高，例如到 $\mathbf{768} \times \mathbf{768}$，以保留原始影像中更多的邊緣和細節資訊。

2.解決小物件與不平衡

- 使用 Focal Loss：替換或結合 CrossEntropyLoss 使用 Focal Loss ，會自動降低對已正確分類的背景或多數類別的權重，使模型訓練更集中於難分類的樣本和極少的目標上。
- 強化 TTA 策略：在推論階段 (Inference) 除了隨機的水平和垂直翻轉外，額外加入 90 度或 180 度旋轉的 TTA 策略。透過多角度的推論結果平均，能進一步提高預測的穩定性和精確度。

3.區分相似類別

- 調整 Loss 權重：重新檢視高度相似但像素數較少的類別權重，確保其在總損失中的貢獻度足夠高，迫使模型學習它們之間的微妙差異。
- 增大上下文感知範圍：擴大 ASPP 的 Dilation Rate 範圍，讓模型能夠依賴更廣泛的全局上下文資訊（例如周圍環境）來區分兩個相似類別，而不僅僅依賴局部紋理。